

Лабораторна робота № 1

З дисципліни «Фреймворк Технології»



**Виконав:**

Студент 3 курсу

3 групи ІПЗ

Сідун Владислав Владиславович

**2026**

## Тема: Вступ до Node.js. Створення простого HTTP-сервера

**Мета:** Ознайомлення із середовищем виконання Node.js, його призначенням та ключовими особливостями. У ході виконання роботи студенти повинні набути практичні навички встановлення та використання Node.js, створення простого HTTP-сервера без використання сторонніх фреймворків.

**Завдання:** Встановити середовище Node.js з використанням NVM (LTS версія). Створити файли .env (з конфігурацією HOSTNAME та PORT) та .gitignore. Реалізувати HTTP-сервер у файлі app.js виключно за допомогою вбудованих модулів. Реалізувати 4 API (GET, POST, PATCH, PUT/DELETE) з валідацією вхідних даних. Дані зберігати лише в пам'яті сервера. Розмістити проєкт у віддаленому репозиторії на гілці lab\_1.

## Виконання

### Варіант 1: Склад комп'ютерної техніки (Inventory)

- Масив: [{ "id": 1, "name": "Monitor", "price": 500, "qty": 10 }]
- GET: Отримати всі товари. Фільтрація за ціною (?minPrice=100).
- POST: Додати новий девайс.
- PATCH: Оновлення по id. Можливість оновити одне або декілька полів, крім id.
- DELETE: Видалити товар.

### App.js

```
const {createServer} = require("node:http");

// Масив зберігається лише в пам'яті
let INVENTORY = [{id: 1, name: "Monitor", price: 500, qty: 10}];

const PORT = process.env.PORT || 3000;
```

```
const HOSTNAME = process.env.HOSTNAME || "localhost";

const server = createServer((req, res) => {
  const method = req.method;
  const parsedUrl = new URL(req.url,
`http://${req.headers.host}`);
  const pathname = parsedUrl.pathname;

  res.setHeader("Content-Type", "application/json; charset=utf-8");

  // GET: Отримати всі товари. Фільтрація за ціною
  (?minPrice=100)
  if (method === "GET" && pathname === "/inventory") {
    const minPrice = parsedUrl.searchParams.get("minPrice");
    let results = [...INVENTORY];

    if (minPrice) {
      const priceFilter = parseFloat(minPrice);
      // Валідація: перевірка чи є параметр числом
      if (isNaN(priceFilter)) {
        res.statusCode = 400;
        return res.end(
          JSON.stringify({error: "Параметр minPrice повинен бути
числом"}),
        );
      }
      results = results.filter((item) => item.price >=
priceFilter);
    }

    res.statusCode = 200;
    return res.end(JSON.stringify({count: results.length, items:
results}));
  }

  // POST: Додати новий девайс
```

```
if (method === "POST" && pathname === "/inventory") {
  let body = "";
  req.on("data", (chunk) => (body += chunk.toString()));
  req.on("end", () => {
    try {
      const data = JSON.parse(body);

      // Валідація вхідних даних
      if (
        !data.name ||
        typeof data.price !== "number" ||
        typeof data.qty !== "number"
      ) {
        res.statusCode = 400;
        return res.end(
          JSON.stringify({
            error:
              "Поля name (рядок), price (число) та qty (число) є обов'язковими.",
          })
        );
      }

      const lastId =
        INVENTORY.length > 0 ? Math.max(...INVENTORY.map((i) => i.id)) : 0;
      const newItem = {
        id: lastId + 1,
        name: data.name,
        price: data.price,
        qty: data.qty,
      };

      INVENTORY.push(newItem);
      res.statusCode = 201;
      return res.end(JSON.stringify({message: "Created", item: newItem}));
    }
  });
}
```

```

    } catch (err) {
      res.statusCode = 400;
      return res.end(JSON.stringify({error: "Невалідний
JSON"}));
    }
  });
  return;
}

// PATCH: Оновлення по id
if (method === "PATCH" && pathname.startsWith("/inventory/")) {
  const id = parseInt(pathname.split("/")[2]);
  if (isNaN(id)) {
    res.statusCode = 400;
    return res.end(JSON.stringify({error: "Некоректний формат
ID"}));
  }

  let body = "";
  req.on("data", (chunk) => (body += chunk.toString()));
  req.on("end", () => {
    try {
      const index = INVENTORY.findIndex((item) => item.id ===
id);
      if (index === -1) {
        res.statusCode = 404;
        return res.end(JSON.stringify({error: "Товар не
знайдено"}));
      }

      const updates = JSON.parse(body);
      delete updates.id; // Заборона оновлення ID

      // Валідація типів, якщо поля передані
      if (updates.price !== undefined && typeof updates.price
!== "number") {
        res.statusCode = 400;

```

```

        return res.end(JSON.stringify({error: "Price повинен
бути числом"}));
    }
    if (updates.qty !== undefined && typeof updates.qty !==
"number") {
        res.statusCode = 400;
        return res.end(JSON.stringify({error: "Qty повинен бути
числом"}));
    }

    INVENTORY[index] = {...INVENTORY[index], ...updates};
    res.statusCode = 200;
    return res.end(
        JSON.stringify({message: "Updated", item:
INVENTORY[index]}),
    );
} catch (err) {
    res.statusCode = 400;
    return res.end(JSON.stringify({error: "Невалідний
JSON"}));
}
});
return;
}

// DELETE: Видалити товар
if (method === "DELETE" && pathname.startsWith("/inventory/"))
{
    const id = parseInt(pathname.split("/")[2]);
    if (isNaN(id)) {
        res.statusCode = 400;
        return res.end(JSON.stringify({error: "Некоректний формат
ID"}));
    }

    const initialLength = INVENTORY.length;
    INVENTORY = INVENTORY.filter((item) => item.id !== id);

```

```

    if (INVENTORY.length < initialLength) {
      res.statusCode = 200;
      return res.end(JSON.stringify({message: "Deleted"}));
    } else {
      res.statusCode = 404;
      return res.end(JSON.stringify({error: "Товар не
знайдено"}));
    }
  }
}

// 404 Route Not Found
res.statusCode = 404;
res.end(JSON.stringify({error: "Маршрут не знайдено"}));
});

server.listen(PORT, HOSTNAME, () => {
  console.log(`Server running at http://${HOSTNAME}:${PORT}/`);
});

```

## Результати виконання

- **Скріншот 1:** Термінал із запуском сервером (node --env-file=.env app.js) та успішним виводом версії nvm list.

```

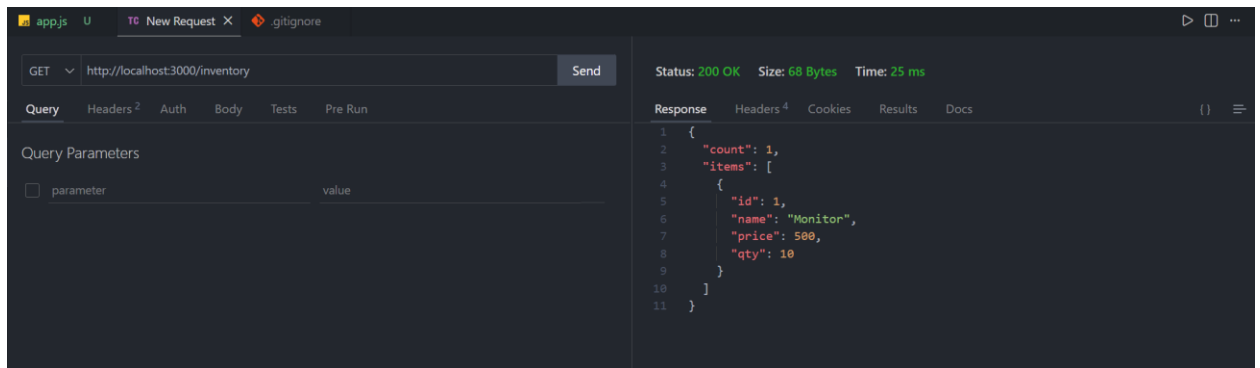
PS C:\Users\vlad0\OneDrive\Робочий стіл\Освіта\UNIVERSITY\6 семестр\Фреймворк\lab_1> nvm list

* 24.14.0 (Currently using 64-bit executable)
  22.13.1

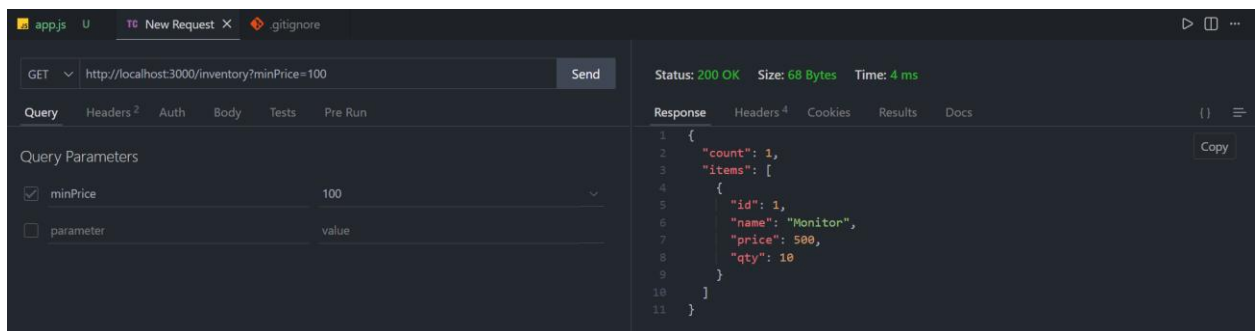
PS C:\Users\vlad0\OneDrive\Робочий стіл\Освіта\UNIVERSITY\6 семестр\Фреймворк\lab_1> node --env-file=.env app.js
Server running at http://localhost:3000/

```

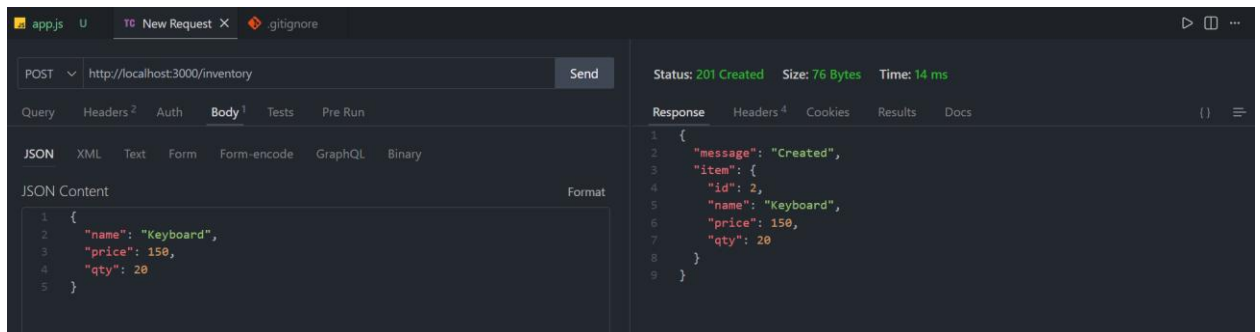
- **Скріншот 2:** Успішний GET запит до /inventory.



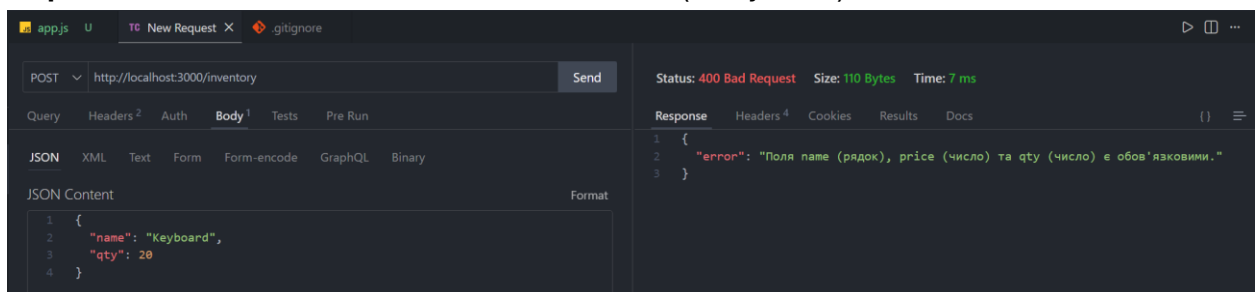
Фільтрація :



- **Скріншот 3:** Успішний POST запит із тілом JSON (додавання товару).

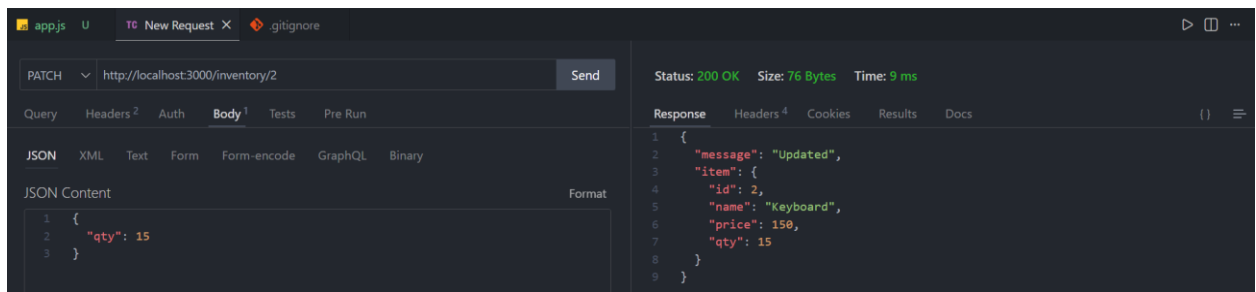


- **Скріншот 4:** POST запит із помилкою валідації (статус 400).

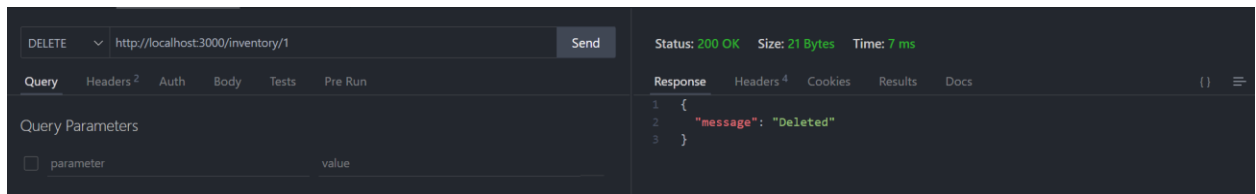


- **Скріншот 5:** Успішний PATCH запит (зміна ціни або кількості).





- **Скріншот 6:** Успішний DELETE запит.



## Посилання на репозиторій

[https://github.com/VladSSidun/FrameWork\\_Technologies/tree/lab\\_1](https://github.com/VladSSidun/FrameWork_Technologies/tree/lab_1)

## Відповіді на контрольні запитання

- 1. Що таке Node.js і для яких задач він використовується?** Node.js — це кросплатформне середовище виконання для мови JavaScript з відкритим вихідним кодом. Він поєднує JIT-компілятор V8 та бібліотеку асинхронного вводу/виводу libuv. Використовується як основна платформа для бекенду (створення REST API, мікросервісів, real-time застосунків) та для інфраструктури фронтенду (утиліти, фреймворки).
- 2. Що таке LTS версія у Node.js?** LTS (Long Term Support) — це версія з довгостроковою підтримкою. Вона є стандартом для комерційної розробки, оскільки гарантує стабільність, сумісність та отримання критичних оновлень безпеки протягом тривалого часу.
- 3. Що таке HTTP-сервер і які його основні функції?** HTTP-сервер — це програмне забезпечення, що очікує на вхідні мережеві з'єднання від клієнтів (наприклад, браузерів) по протоколу HTTP. Основні функції: розбір вхідного запиту, маршрутизація (визначення, що саме запитує клієнт), виконання відповідної бізнес-логіки та формування зворотної відповіді зі статус-кодом.

**4. Яке призначення вбудованого модуля http у Node.js?** Модуль node:http — це низькорівневий інструмент для створення вебсерверів та клієнтів. Він надає абстракції для керування HTTP-заголовками, статус-кодами та поетапною передачею тіла повідомлень (через потоки).

**5. Що таке порт і чому сервер не може працювати без нього?** Порт — це числовий ідентифікатор логічного каналу в операційній системі (від 1 до 65535). Сервер не може працювати без нього, оскільки на одному хості (IP-адресі) може одночасно працювати багато мережових програм. Порт вказує операційній системі, якому саме процесу (в даному випадку нашому Node.js серверу) потрібно передати вхідний мережовий запит.

**6. Що відбувається під час HTTP-запиту від клієнта до сервера?** Клієнт ініціює TCP-з'єднання з сервером за вказаним IP та портом і передає текстове повідомлення (заголовки та тіло). Сервер на базі Node.js приймає запит, делегуючи мережові операції на рівень ядра ОС (або libuv), а потім передає об'єкти запиту та відповіді у Callback-функцію Event Loop-а для виконання розробником бізнес-логіки. Після формування відповіді з'єднання закривається (або підтримується через keep-alive).

**7. Що таке request і response у контексті HTTP-сервера?** request (req) — об'єкт, який містить усі дані вхідного повідомлення від клієнта (HTTP-метод, URL, параметри шляху, заголовки та потік вхідних даних) . response (res) — об'єкт, за допомогою якого сервер конструює і надсилає відповідь клієнту (встановлює statusCode, заголовки setHeader та відправляє кінцеві дані через метод end).

**8. Яка команда використовується для перевірки версії Node.js?** Команда node -v.

**9. Як задати HTTP-статус код у відповіді сервера?** За допомогою властивості statusCode об'єкта response. Наприклад: res.statusCode = 200; для успішного виконання або res.statusCode = 404; якщо ресурс не знайдено.

**10. Яку функцію виконує метод createServer()?** Метод createServer() створює екземпляр HTTP-сервера. Він приймає функцію-слухача (callback), яка буде автоматично викликатися системою при кожному новому HTTP-запиті, передаючи в неї екземпляри об'єктів вхідного запиту (req) та вихідної відповіді (res).